PrismTemplate

October 16, 2026

[32]:
```
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
def cauchy(wavelength, a, b):
    return a + b / wavelength**2
```

For the analysis of this lab, you will be fitting your data to the Cauchy's Dispersion Equation which is a non-polynomial equation. In order to complete this fit we will have to include and additional function from the scipy library called curve_fit.

Syntax: *from scipy.optimize import curve_fit*

[10]:
```
#Import curve fit from scipy
from scipy.optimize import curve_fit
```

[11]:
```
#Import data
datafile = pd.read_excel('Refractive Index Measurements.xlsx',
                         sheet_name = 'Sheet1')

#Display data
display(datafile)
```

```
                                Unnamed: 0                    Unnamed: 1  \
0                                      NaN                           NaN
1                                      NaN  Relative Intesity (out of 10)
2                                      Red                             2
3                                   Orange                             4
4                                    Green                             9
5                                     Cyan                             1
6                              Deep Purple                             5
7                            Deeper Purple              (Not Available)
8                                      NaN                           NaN
9  Step 16 Thickness of slit = 1.6 +/-0.02mm                           NaN

  Minimum Angles Trial 1 Unnamed: 3 Unnamed: 4 Unnamed: 5  \
0                Theta 1        NaN    Theta 2        NaN
1                Degrees    Minutes    Degrees    Minutes
2                    336         40        241         37
```

```
3                    337         30        240         50
4                    337         48        240         32
5                    338         31        239         48
6                    339         35        238         43
7                    340         28        337         48
8                    NaN        NaN        NaN        NaN
9                    NaN        NaN        NaN        NaN

  Minimum Angles Trial 2 Unnamed: 7 Unnamed: 8 Unnamed: 9  \
0                Theta 1        NaN    Theta 2        NaN
1                Degrees    Minutes    Degrees    Minutes
2                    336         38        241         36
3                    337         31        241         51
4                    337         49        240         31
5                    338         32        239         50
6                    339         34        238         44
7                    340         31        239         50
8                    NaN        NaN        NaN        NaN
9                    NaN        NaN        NaN        NaN

  Minimum Angles Trial 3 Unnamed: 11 Unnamed: 12 Unnamed: 13
0                Theta 1         NaN     Theta 2         NaN
1                Degrees     Minutes     Degrees     Minutes
2                    336          39         241          36
3                    337          30         240          50
4                    337          49         240          32
5                    338          31         239          47
6                    339          36         238          44
7                    340          30         237          52
8                    NaN         NaN         NaN         NaN
9                    NaN         NaN         NaN         NaN
```

[34]:
```python
#Convert data from list to array
degrees = datafile['Minimum Angles Trial 1'].iloc[3:8].to_numpy()
minutes = datafile['Unnamed: 3'].iloc[3:8].to_numpy()
degrees2 = datafile['Unnamed: 4'].iloc[3:8].to_numpy()
minutes2 = datafile['Unnamed: 5'].iloc[3:8].to_numpy()


# Convert both arrays to numeric arrays
degrees = degrees.astype(float)
minutes = minutes.astype(float)
degrees2 = degrees2.astype(float)
minutes2 = minutes2.astype(float)

#Convert angle values from arcminutes to deg, and deg to rad
angle_deg = degrees + minutes / 60
```

```
angle_rad = np.deg2rad(angle_deg)

angle_deg2 = degrees2 + minutes2 / 60
angle_rad2 = np.deg2rad(angle_deg2)
print(angle_deg)  # [30.25 45.5  60.75]
print(angle_deg2)  # [30.25 45.5  60.75]

#Determine the angle of deviation to each spectral line
deviation_deg = (angle_deg - angle_deg2)/2
deviation_rad = (angle_rad - angle_rad2)/2

colours = ["Red", "Orange", "Green", "Cyan",
           "Deep Purple", "Deeper Purple"]

for colour, deviation in zip(colours, deviation_deg):
    print(f"{colour}: {deviation:.3f} degrees")

#Convert apex angle from deg to rad
apex_rad = np.deg2rad(60)

#Converting reading uncertainty to rad
deviation_uncertainty_rad = np.deg2rad(0.5 / 60)

#Calculate the refractive index of the prism and its uncertainty
#Uncertainty
dn_dF = 0.5 * np.cos((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)


#Refractive index for each spectral line
n = np.sin((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)

print("The n's are ", n)

#the errors
import numpy as np

# Apex angle and its uncertainty from the manual
apex_rad = np.deg2rad(60.063)
apex_uncertainty_rad = np.deg2rad(0.029)

# Assuming each telescope reading has uncertainty ±0.5 arcminutes
reading_uncertainty_rad = np.deg2rad(0.5 / 60)

# For deviation = abs(theta1 - theta2) / 2,
# assuming the two reading uncertainties are independent
deviation_uncertainty_rad = (
    np.sqrt(reading_uncertainty_rad**2
```

```
            + reading_uncertainty_rad**2) / 2
)

# Your existing array of minimum deviations, converted to radians
deviation_rad = np.deg2rad(deviation_deg)

# Refractive index for each colour
n = np.sin((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)

# Derivatives with respect to deviation and apex angle
dn_ddelta = (
    np.cos((apex_rad + deviation_rad) / 2)
    / (2 * np.sin(apex_rad / 2))
)

dn_dA = (
    -np.sin(deviation_rad / 2)
    / (2 * np.sin(apex_rad / 2)**2)
)

# Combine the uncertainty contributions
n_uncertainty = np.sqrt(
    (dn_ddelta * deviation_uncertainty_rad)**2
    + (dn_dA * apex_uncertainty_rad)**2
)

for colour, value, uncertainty in zip(colours, n, n_uncertainty):
    print(f"{colour}: n = {value:.6f} ± {uncertainty:.6f}")
```

```
[337.5        337.8        338.51666667 339.58333333 340.46666667]
[240.83333333 240.53333333 239.8        238.71666667 337.8       ]
Red: 48.333 degrees
Orange: 48.633 degrees
Green: 49.358 degrees
Cyan: 50.433 degrees
Deep Purple: 1.333 degrees
The n's are  [1.62144673 1.62450648 1.63185484 1.64263038 1.02008518]
Red: n = 1.620547 ± 0.000418
Orange: n = 1.623602 ± 0.000420
Green: n = 1.630938 ± 0.000426
Cyan: n = 1.641694 ± 0.000434
Deep Purple: n = 1.020060 ± 0.000089
```

[36]:
```
#Convert data from list to array
degrees = datafile['Minimum Angles Trial 1'].iloc[3:8].to_numpy()
minutes = datafile['Unnamed: 3'].iloc[3:8].to_numpy()
degrees2 = datafile['Unnamed: 4'].iloc[3:8].to_numpy()
```

```
minutes2 = datafile['Unnamed: 5'].iloc[3:8].to_numpy()


# Convert both arrays to numeric arrays
degrees = degrees.astype(float)
minutes = minutes.astype(float)
degrees2 = degrees2.astype(float)
minutes2 = minutes2.astype(float)

#Convert angle values from arcminutes to deg, and deg to rad
angle_deg = degrees + minutes / 60
angle_rad = np.deg2rad(angle_deg)

angle_deg2 = degrees2 + minutes2 / 60
angle_rad2 = np.deg2rad(angle_deg2)
print(angle_deg)  # [30.25 45.5  60.75]
print(angle_deg2)  # [30.25 45.5  60.75]

#Determine the angle of deviation to each spectral line
deviation_deg = (angle_deg - angle_deg2)/2
deviation_rad = (angle_rad - angle_rad2)/2

colours = ["Red", "Orange", "Green", "Cyan",
           "Deep Purple", "Deeper Purple"]

for colour, deviation in zip(colours, deviation_deg):
    print(f"{colour}: {deviation:.3f} degrees")

#Convert apex angle from deg to rad
apex_rad = np.deg2rad(60)

#Converting reading uncertainty to rad
deviation_uncertainty_rad = np.deg2rad(0.5 / 60)

#Calculate the refractive index of the prism and its uncertainty
#Uncertainty
dn_dF = 0.5 * np.cos((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)


#Refractive index for each spectral line
n = np.sin((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)

print("The n's are ", n)

#the errors
import numpy as np
```

```python
# Apex angle and its uncertainty from the manual
apex_rad = np.deg2rad(60.063)
apex_uncertainty_rad = np.deg2rad(0.029)

# Assuming each telescope reading has uncertainty ±0.5 arcminutes
reading_uncertainty_rad = np.deg2rad(0.5 / 60)

# For deviation = abs(theta1 - theta2) / 2,
# assuming the two reading uncertainties are independent
deviation_uncertainty_rad = (
    np.sqrt(reading_uncertainty_rad**2
            + reading_uncertainty_rad**2) / 2
)

# Your existing array of minimum deviations, converted to radians
deviation_rad = np.deg2rad(deviation_deg)

# Refractive index for each colour
n = np.sin((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)

# Derivatives with respect to deviation and apex angle
dn_ddelta = (
    np.cos((apex_rad + deviation_rad) / 2)
    / (2 * np.sin(apex_rad / 2))
)

dn_dA = (
    -np.sin(deviation_rad / 2)
    / (2 * np.sin(apex_rad / 2)**2)
)

# Combine the uncertainty contributions
n_uncertainty = np.sqrt(
    (dn_ddelta * deviation_uncertainty_rad)**2
    + (dn_dA * apex_uncertainty_rad)**2
)

for colour, value, uncertainty in zip(colours, n, n_uncertainty):
    print(f"{colour}: n = {value:.6f} ± {uncertainty:.6f}")
```

```
[337.5        337.8        338.51666667 339.58333333 340.46666667]
[240.83333333 240.53333333 239.8        238.71666667 337.8       ]
Red: 48.333 degrees
Orange: 48.633 degrees
Green: 49.358 degrees
Cyan: 50.433 degrees
Deep Purple: 1.333 degrees
The n's are  [1.62144673 1.62450648 1.63185484 1.64263038 1.02008518]
```

```
Red: n = 1.620547 ± 0.000418
Orange: n = 1.623602 ± 0.000420
Green: n = 1.630938 ± 0.000426
Cyan: n = 1.641694 ± 0.000434
Deep Purple: n = 1.020060 ± 0.000089
```

[35]:
```python
#Convert data from list to array
degrees = datafile['Minimum Angles Trial 3'].iloc[3:8].to_numpy()
minutes = datafile['Unnamed: 11'].iloc[3:8].to_numpy()
degrees2 = datafile['Unnamed: 12'].iloc[3:8].to_numpy()
minutes2 = datafile['Unnamed: 13'].iloc[3:8].to_numpy()


# Convert both arrays to numeric arrays
degrees = degrees.astype(float)
minutes = minutes.astype(float)
degrees2 = degrees2.astype(float)
minutes2 = minutes2.astype(float)

#Convert angle values from arcminutes to deg, and deg to rad
angle_deg = degrees + minutes / 60
angle_rad = np.deg2rad(angle_deg)

angle_deg2 = degrees2 + minutes2 / 60
angle_rad2 = np.deg2rad(angle_deg2)
print(angle_deg)  # [30.25 45.5  60.75]
print(angle_deg2)  # [30.25 45.5  60.75]

#Determine the angle of deviation to each spectral line
deviation_deg = (angle_deg - angle_deg2)/2
deviation_rad = (angle_rad - angle_rad2)/2

colours = ["Red", "Orange", "Green", "Cyan",
           "Deep Purple", "Deeper Purple"]

for colour, deviation in zip(colours, deviation_deg):
    print(f"{colour}: {deviation:.3f} degrees")

#Convert apex angle from deg to rad
apex_rad = np.deg2rad(60)

#Converting reading uncertainty to rad
deviation_uncertainty_rad = np.deg2rad(0.5 / 60)

#Calculate the refractive index of the prism and its uncertainty
#Uncertainty
dn_dF = 0.5 * np.cos((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)
```

```
#Refractive index for each spectral line
n = np.sin((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)

print("The n's are ", n)

#the errors
import numpy as np

# Apex angle and its uncertainty from the manual
apex_rad = np.deg2rad(60.063)
apex_uncertainty_rad = np.deg2rad(0.029)

# Assuming each telescope reading has uncertainty ±0.5 arcminutes
reading_uncertainty_rad = np.deg2rad(0.5 / 60)

# For deviation = abs(theta1 - theta2) / 2,
# assuming the two reading uncertainties are independent
deviation_uncertainty_rad = (
    np.sqrt(reading_uncertainty_rad**2
            + reading_uncertainty_rad**2) / 2
)

# Your existing array of minimum deviations, converted to radians
deviation_rad = np.deg2rad(deviation_deg)

# Refractive index for each colour
n = np.sin((apex_rad + deviation_rad) / 2) / np.sin(apex_rad / 2)

# Derivatives with respect to deviation and apex angle
dn_ddelta = (
    np.cos((apex_rad + deviation_rad) / 2)
    / (2 * np.sin(apex_rad / 2))
)

dn_dA = (
    -np.sin(deviation_rad / 2)
    / (2 * np.sin(apex_rad / 2)**2)
)

# Combine the uncertainty contributions
n_uncertainty = np.sqrt(
    (dn_ddelta * deviation_uncertainty_rad)**2
    + (dn_dA * apex_uncertainty_rad)**2
)
```

```
for colour, value, uncertainty in zip(colours, n, n_uncertainty):
    print(f"{colour}: n = {value:.6f} ± {uncertainty:.6f}")
```

```
[337.5        337.81666667 338.51666667 339.6        340.5       ]
[240.83333333 240.53333333 239.78333333 238.73333333 237.86666667]
Red: 48.333 degrees
Orange: 48.642 degrees
Green: 49.367 degrees
Cyan: 50.433 degrees
Deep Purple: 51.317 degrees
The n's are  [1.62144673 1.62459131 1.63193893 1.64263038 1.65137654]
Red: n = 1.620547 ± 0.000418
Orange: n = 1.623686 ± 0.000420
Green: n = 1.631021 ± 0.000426
Cyan: n = 1.641694 ± 0.000434
Deep Purple: n = 1.650425 ± 0.000441
```

[21]:
```
#Define a figure so that you can plot your line of best fit over your data␣
 ↪later in the file
import matplotlib.pyplot as plt

# Define a figure and axes for your data and best-fit line
fig, ax = plt.subplots()

ax.errorbar(
    x, y,
    xerr=x_err,
    yerr=y_err,
    fmt='o',
    capsize=3,
    label='Measured data'
)

ax.legend()
#x-axis title
#y-axis title
#Figure title
```

To use the curve fit function from scipy to fit the Cauchy Equation to your data, you will fit need to define the function and the parameters that you would like to fit.

Syntax: *def Cauchy(x, a, b, c): return a + b/pow(x,2) + c/pow(x, 4)*

In the syntax above, we first define what we will call the function (in this case Cauchy) as well as the fiting variable (x) and the fitting parameters (a, b, and c). We then define the equation the will be returned when we call the Cauchy function. This equation will be the Cauchy Equation in the case of this lab, Eqn 6 in the lab manual.

NOTE: You can call your functions anything, and you can define whatever equation you would like (ie, sine, exponential, etc). The example above is to guide you through the fit for this lab, but

in future labs, you will need to define other functions.

[22]:
```
#Define fit function for Cauchy relationship using Scipy curve fit
from scipy.optimize import curve_fit

def cauchy(wavelength, a, b):
    return a + b / wavelength**2
```

If python is stuggling to complete the fit properly, you can provide initial guesses for the algorithm to use as starting points.

Syntax: *ParameterStartingPoints = numpy.array([ParameterAGuess, ParameterBGuess, ParameterCGuess])*

[23]:
```
#Provide initial guesses to the fit
ParameterStartingPoints = [1.5, 10000]
```

The fit itself will be completed in a similar way the we completed the polyfit. You will define the fit parameters, and the covariance array, then use the curve_fit fuction to fit your defined function to the data.

Syntax: *FitParameters, CovMatrix = curve_fit(DefineFitFunction, x-data, y-data)*

If you need to use your initial guess array to help the algorithm, you can add a fourth argument to the fit function.

Syntax: *FitParameters, CovMatrix = curve_fit(DefineFitFunction, x-data, y-data, InitialGuessArray)*

Finally, if you want curve_fit's algorithm to consider the uncertainties of the y-values, you can include them as the fifth argument to the function:

Syntax: *FitParameters, CovMatrix = curve_fit(DefineFitFunction, x-data, y-data, InitialGuessArray, sigma)*

For more information about the curve_fit function, see the SciPy documentation.

After completing the fit, you can extract the fit parameters and their uncertainties the same way as when polyfit was used.

[39]:
```
#Perform the fit on the data using the defined function
#FitParameters, CovMatrix =

# Starting guesses for a, b, c
ParameterStartingPoints = [1.5, 1e4, 1e8]

# Reference wavelengths in nm:
# Yellow 1, Yellow 2, Green, Blue-violet, Violet
wavelengths = np.array([
    579.0663,
    576.9598,
    546.0735,
```

```
    435.8328,
    404.6563
])
x = np.array(wavelengths, dtype=float)
FitParameters, CovMatrix = curve_fit(
    cauchy,
    x,
    n,
    p0=ParameterStartingPoints,
    sigma=n_uncertainty,
    absolute_sigma=True
)
FitUncertainty = np.sqrt(np.diag(CovMatrix))


#Assign the fit variables
a, b = FitParameters
a_err, b_err = FitUncertainty
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[39], line 17
      9 wavelengths = np.array([
     10     579.0663,
     11     576.9598,
   (…)     14     404.6563
     15 ])
     16 x = np.array(wavelengths, dtype=float)
---> 17 FitParameters, CovMatrix = curve_fit(
     18     cauchy,
     19     x,
     20     n,
     21     p0=ParameterStartingPoints,
     22     sigma=n_uncertainty,
     23     absolute_sigma=True
     24 )
     25 FitUncertainty = np.sqrt(np.diag(CovMatrix))
     28 #Assign the fit variables

File /usr/local/lib/python3.13/site-packages/scipy/optimize/_minpack_py.py:1021
 ↪in curve_fit(f, xdata, ydata, p0, sigma, absolute_sigma, check_finite, bounds
 ↪method, jac, full_output, nan_policy, **kwargs)
   1018 if ydata.size != 1 and n > ydata.size:
   1019     raise TypeError(f"The number of func parameters={n} must not"
   1020                     f" exceed the number of data points={ydata.size}")
-> 1021 res = leastsq(func, p0, Dfun=jac, full_output=1, **kwargs)
   1022 popt, pcov, infodict, errmsg, ier = res
```

```
   1023 ysize = len(infodict['fvec'])

File /usr/local/lib/python3.13/site-packages/scipy/optimize/_minpack_py.py:426,
 ↪in leastsq(func, x0, args, Dfun, full_output, col_deriv, ftol, xtol, gtol,␣
 ↪maxfev, epsfcn, factor, diag)
    424 if not isinstance(args, tuple):
    425     args = (args,)
--> 426 shape, dtype = _check_func(          ,        , func, x0, args, n)
    427 m = shape[0]
    429 if n > m:

File /usr/local/lib/python3.13/site-packages/scipy/optimize/_minpack_py.py:24,␣
 ↪in _check_func(checker, argname, thefunc, x0, args, numinputs, output_shape)
     22 def _check_func(checker, argname, thefunc, x0, args, numinputs,
     23                 output_shape=None):
---> 24     res = atleast_1d(thefunc(*((x0[:numinputs],) + args)))
     25     if (output_shape is not None) and (shape(res) != output_shape):
     26         if (output_shape[0] != 1):

File /usr/local/lib/python3.13/site-packages/scipy/optimize/_minpack_py.py:523,
 ↪in _lightweight_memoizer.<locals>._memoized_func(params)
    520 elif _memoized_func.last_params is not None:
    521     _memoized_func.skip_lookup = True
--> 523 val = f(params)
    525 if _memoized_func.last_params is None:
    526     _memoized_func.last_params = np.copy(params)

File /usr/local/lib/python3.13/site-packages/scipy/optimize/_minpack_py.py:543,
 ↪in _wrap_func.<locals>.func_wrapped(params)
    542 def func_wrapped(params):
--> 543     return transform * (func(xdata, *params) - ydata)

TypeError: cauchy() takes 3 positional arguments but 4 were given
```

Since you have fined a function to complete your fit instead of using poly fit, you can use the function to quickly find the y-values of your fit as well.

Syntax: *yFitValues = function(x-values,* FitParameters)

This calculates the values for the function that was defines given each x-value from data, and each parameter value that was obtained from the fit.

[25]:
```
#Determine y-values of the lines of best fit using the fit parameters
yFitValues =
```

[26]:
```
#plot line of best fit on the same figure as your data. Again, provide a label␣
 ↪for the legend.
```

```
#add a legend
#print figure
```

[27]:
```
#Save a copy of the graph.
```

[28]:
```
#Calculate chi squared for the fit that was performed


#Print chi squared value
```

0.1 Determine Dispersive Power

[29]:
```
#Determine the index of refraction for red light and its error


#Determine the index of refraction for yellow light and its error


#Determine the index of refraction for blue light and its error


#Determine the dispersive power of the prism and its error


#Print values
```

[30]:
```
#Determine the Abbes number for the prism and its error


#Print values
```

0.2 Determine the Resolving Power

[31]:
```
# S is the measure beam width that hits the prism
S =
SErr =

#Average wavelength of sodium doublet


#Determine the change in refractive index with repect to change in wavelength␣
 ↪and its error


#Determine the resolving power of the prism and its error
```

```
#Determine the minimum seperable wavelengths given the resolving power above␣
 ↪and its error


#Print values
```